

ALCANCE DEL PROYECTO FINAL DEL CURSO ESTRUCTURAS DE DATOS Y ALGORITMOS II.

Sarai Isajar Zuniga¹, Jhojan Alexander Valencia Torres², Stiven Ortiz Muñoz³

sarai.isajar@uao.edu.co¹; jhojan.valencia@uao.edu.co²; stiven.ortiz@uao.edu.co³,
Universidad Autónoma de Occidente ^{1, 2, 3 y 4.}

1. ALCANCE DEL SISTEMA

El sistema desarrollado consiste en una plataforma web interactiva de tipo E-Commerce optimizada para la gestión, simulación y adquisición de licencias digitales de software y videojuegos. El alcance abarca desde la interfaz de usuario de cara al cliente hasta la arquitectura de datos en memoria para soportar operaciones de alta velocidad con baja latencia.

Módulos principales incluidos en el sistema:

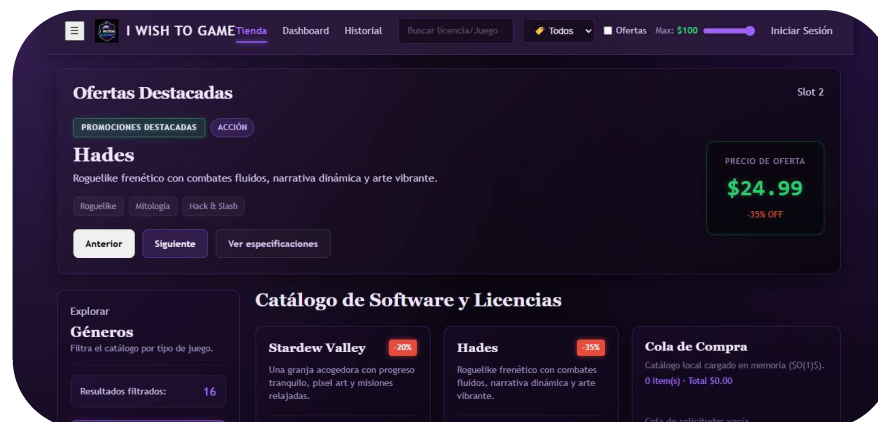
Módulo de Navegación y Catálogo Interactivo: Renderizado dinámico de productos mediante una interfaz Cyberpunk/Glassmorphic altamente responsiva, organizada por árboles de categorías y etiquetas dinámicas.

Sistema de Filtrado Multitarea en Tiempo Real: Motor de búsqueda integrado en la barra superior que permite discriminar productos por texto concurrente, rangos de precio máximos, etiquetas (tags) específicas y estado de ofertas vigentes de forma simultánea.

Panel de Promociones Destacadas (Featured Carousel): Carrusel de visualización asíncrona que destaca las licencias prioritarias y con descuentos activos, interactuando directamente con el catálogo global.

Gestor de Cola de Compra (Carrito de Compras en Memoria): Sistema persistente durante la sesión que administra las solicitudes de compra del usuario, calculando instantáneamente el total de ítems y el costo acumulado de las licencias.

Módulo de Historial de Adquisiciones: Panel dedicado que registra la pila cronológica de las transacciones simuladas por el usuario, permitiendo la trazabilidad de las licencias obtenidas con opciones de reversión de operaciones (quitar último elemento).



2. ECOSISTEMA TECNOLÓGICO

Para garantizar un desarrollo modular, tipado seguro y estilos aislados de alto rendimiento visual, se seleccionó el siguiente stack tecnológico:

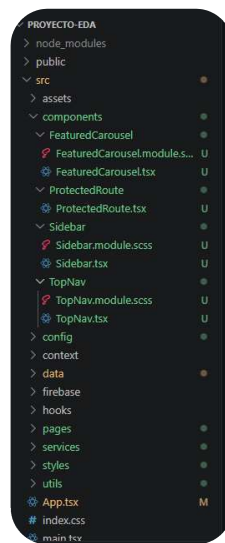
React (v18+) & TypeScript (v5+): Utilizado como la librería núcleo para la construcción de la interfaz basada en componentes declarativos. TypeScript actúa como capa de tipado estricto, mitigando errores en tiempo de compilación mediante contratos de interfaces claros (ej. TopNavProps), garantizando la integridad de los datos que fluyen entre rutas y componentes.

Vite: Seleccionado como herramienta de empaquetado y entorno de desarrollo rápido, ofreciendo Hot Module Replacement (HMR) instantáneo para agilizar las fases de maquetación y pruebas de código.

Sass (Syntactically Awesome Style Sheets) - SCSS Modules: Implementado mediante arquitectura de módulos independientes (.module.scss). Esto asegura el aislamiento absoluto de los estilos por componente (CSS Scoping), evitando colisiones globales. Se utilizaron características avanzadas de Sass como mixins personalizados para el renderizado del efecto traslúcido (glassmorphic) y mapeo de variables para la paleta neón Cyberpunk de la app.

React Router DOM (v6+): Motor encargado del enrutamiento del lado del cliente (Client-Side Routing), gestionando los cambios de estado de las páginas (/ , /history, /game/:id, /dashboard) de forma asíncrona sin recargar el navegador, emulando la experiencia de una Single Page Application (SPA).

React Context API: Utilizado para la gestión del estado global de la aplicación a través de proveedores especializados (AuthContext, CartContext). Centraliza la lógica del carrito y la sesión del usuario para proveer acceso instantáneo a cualquier nivel del árbol de componentes sin incurrir en prop drilling.



3. ESTRUCTURAS DE DATOS UTILIZADAS

Las operaciones de la aplicación web (búsquedas rápidas, jerarquías de navegación y procesamiento ordenado de transacciones), se implementaron 5 estructuras de datos fundamentales. A continuación, se detalla su comportamiento, complejidad algorítmica:

- **Lista Dinámica Indexada (Array de Objetos de Catálogo)**, Se compone de una colección secuencial y contigua en memoria que almacena instancias estructuradas del tipo de dato Game. Permite el acceso indexado y la iteración directa a través como .map(), .filter() y .find().

El catálogo de licencias es una estructura que se lee constantemente, pero se modifica de manera poco frecuente en el cliente. Un array dinámico es ideal porque ofrece una sobrecarga de memoria (*overhead*) mínima y aprovecha los métodos nativos de optimización del motor V8 del navegador para iterar y renderizar la cuadrícula del catálogo de forma fluida.

- **Árbol N-ario Jerárquico (N-ary Tree)**, Un árbol donde cada nodo (CategoryNode) representa una categoría de la tienda, conteniendo propiedades de identidad (title, link) y un arreglo de sub-nodos hijos (children?).

CategoryNode[]). Al no limitarse a un máximo de dos bifurcaciones por nodo, se clasifica formalmente como un árbol no binario o N-ario.

La categorización de un e-commerce es inherentemente jerárquica (ej. Acción -> Roguelike / Soulslike). Un árbol N-ario permite representar esta taxonomía de manera natural y recursiva, facilitando que el componente Sidebar dibuje menús desplegables anidados sin duplicar información y manteniendo un orden lógico de dependencia.

- Cola FIFO Lineal (First-In, First-Out Purchase Queue), Estructura de datos abstracta y secuencial que sigue estrictamente el principio de que el primer elemento en ingresar es el primero en ser procesado. Utiliza un arreglo interno encapsulado, exponiendo formalmente los métodos de control primitivos: enqueue() (inserción al final usando .push()) y dequeue() (extracción del inicio usando .shift()).

Durante la función de checkout(), la aplicación debe procesar las licencias adquiridas de manera secuencial y ordenada. La cola garantiza un flujo determinista: procesa los videojuegos exactamente en el orden en que el usuario los revisó en su carrito, impidiendo condiciones de carrera o saltos anómalos en el procesamiento asíncrono de pedidos de cara al backend.

- Registro Estructurado (Hash Map / Object Literal de Estado), Estructura basada en pares clave-valor (Key-Value) donde cada propiedad del elemento del carrito está mapeada a un espacio de memoria direccionable. Se utiliza para verificar de manera unívoca si un producto ya existe en la cola de compra mediante su identificador.

Al agregar un elemento al carrito (addToCart), es ineficiente duplicar renglones si el usuario quiere más de una licencia del mismo juego. El registro permite evaluar el objeto instantáneamente, mutar la propiedad quantity: item.quantity + 1 en tiempo constante si se localiza el ID, manteniendo la consistencia de la persistencia de datos.

- Pila de Historial y Estado Reactivo, Colección secuencial gobernada bajo el principio de inmutabilidad de React. Cada modificación en el carrito no muta la estructura original, sino que "apila" un nuevo estado de la memoria en el ciclo de vida del componente mediante propagación por esparcimiento ([...cartItems, newItem]), permitiendo además operaciones tipo LIFO (Last-In, First-Out) al revertir transacciones en páginas complementarias.

La arquitectura reactiva de la aplicación requiere que el estado sea predecible e inmutable. Utilizar una pila de estados clonados garantiza que los componentes visuales (TopNav, FeaturedCarousel) se enteren de manera síncrona e instantánea de los cambios de la cola de compra, protegiendo la integridad visual de la app frente a interrupciones concurrentes.

4. CONCLUSIÓN

El desarrollo de la plataforma consolidó con éxito la integración de una interfaz de usuario moderna de alto impacto visual con una arquitectura de software robusta, predecible y eficiente.

La implementación del Sass Modules demostró que un diseño visualmente complejo y avanzado no requiere sacrificar el rendimiento ni el orden del código. El aislamiento de estilos por componente garantizó una maquetación escalable, libre de colisiones globales y altamente mantenible.

El uso de TypeScript como capa de tipado estricto fue determinante para mitigar errores lógicos en tiempo de desarrollo. La definición rigurosa de contratos de datos blindó la comunicación entre rutas y componentes, garantizando que el flujo de información por el ecosistema de React fuera predecible y consistente ante cualquier refactorización.

La resolución de los requerimientos de la tienda mediante estructuras de datos específicas como el uso de un Árbol N-ario para la navegación jerárquica de categorías, una Cola FIFO nativa para el procesamiento secuencial y



ordenado del checkout, y colecciones dinámicas inmutables para el estado global del carrito probó que el éxito de una aplicación web depende directamente de emparejar cada problema lógico con la estructura de datos algorítmicamente óptima.

El aprovechamiento de la Context API de React permitió centralizar las reglas de negocio críticas sin incurrir en malas prácticas. Esto resultó en una experiencia de usuario fluida, reactiva e instantánea, donde cada módulo de la aplicación coexiste y reacciona de manera unificada y síncrona en tiempo real.

El proyecto no solo cumple con los requerimientos funcionales de un e-commerce simulado, sino que se erige como un modelo de desarrollo robusto, limpio y escalable, alineado con los estándares modernos de la ingeniería de software en el ecosistema de JavaScript/TypeScript.